

# Komplexaufgabe zur Algorithmmierung

Informatik · Klasse 9 · Praesentationen

# Inhaltsverzeichnis

|                                            |           |
|--------------------------------------------|-----------|
| <b>Präsentationen</b>                      | <b>3</b>  |
| <b>Lineare Algorithmen</b>                 | <b>4</b>  |
| Leitfrage . . . . .                        | 4         |
| <b>Das Problem</b>                         | <b>5</b>  |
| Frage . . . . .                            | 5         |
| <b>KAI ergänzt nichts</b>                  | <b>6</b>  |
| <b>Gute Algorithmen</b>                    | <b>7</b>  |
| <b>Merksatz</b>                            | <b>8</b>  |
| <b>Mensch und Computer</b>                 | <b>9</b>  |
| <b>Algorithmus und Programmiersprache</b>  | <b>10</b> |
| <b>Scratch</b>                             | <b>11</b> |
| <b>Python</b>                              | <b>12</b> |
| <b>Was bleibt gleich?</b>                  | <b>13</b> |
| <b>Was ändert sich?</b>                    | <b>14</b> |
| <b>Entwickeln heißt testen</b>             | <b>15</b> |
| <b>Abschluss</b>                           | <b>16</b> |
| Kannst du ... . . . . .                    | 16        |
| <b>Komplexaufgabe zur Algorithmmierung</b> | <b>17</b> |
| Unser Ziel . . . . .                       | 17        |
| <b>Nicht sofort programmieren</b>          | <b>18</b> |
| <b>Projektstart</b>                        | <b>19</b> |
| Drei Fragen . . . . .                      | 19        |
| <b>Muss oder Kann?</b>                     | <b>20</b> |
| Muss . . . . .                             | 20        |
| Kann . . . . .                             | 20        |
| <b>Gute Anforderungen</b>                  | <b>21</b> |
| <b>EVAS nutzen</b>                         | <b>22</b> |
| <b>Mehr als eine Idee</b>                  | <b>23</b> |
| <b>Version 1</b>                           | <b>24</b> |
| Klein anfangen . . . . .                   | 24        |
| <b>Inkrementell entwickeln</b>             | <b>25</b> |
| <b>Iterativ verbessern</b>                 | <b>26</b> |
| <b>Testfall</b>                            | <b>27</b> |

|                              |           |
|------------------------------|-----------|
| <b>Fremdtest</b>             | <b>28</b> |
| Regel . . . . .              | 28        |
| <b>Fehler priorisieren</b>   | <b>29</b> |
| <b>Dokumentieren</b>         | <b>30</b> |
| <b>Präsentieren</b>          | <b>31</b> |
| <b>Abschlussfrage</b>        | <b>32</b> |
| Was ist wichtiger? . . . . . | 32        |

# Präsentationen

Die Präsentationen unterstützen den Unterricht, ersetzen aber nicht das Arbeitsheft.

Die Markdown-Dateien sind als Folienquellen gedacht und können später automatisiert in Präsentationsformate überführt werden.

# Lineare Algorithmen

## Leitfrage

Wie muss eine Anweisung aussehen, damit KAI sie ohne Rückfragen ausführen kann?

---

## Das Problem

1. Rühre gut um.
2. Gib zwei Teelöffel Kakao dazu.
3. Fülle Milch in einen Becher.

## Frage

Was passiert, wenn KAI diese Anleitung exakt ausführt?

---

## KAI ergänzt nichts

KAI

„Was soll ich in Schritt 1 umrühren?“

---

# Gute Algorithmen

Sie sind:

- eindeutig,
  - vollständig,
  - ausführbar,
  - sinnvoll geordnet.
-

## Merksatz

Ein Algorithmus ist eine eindeutige, vollständige und ausführbare Folge von Handlungsschritten zur Lösung eines Problems.

---

# Mensch und Computer

Menschen:

- nutzen Erfahrungen,
- ergänzen Zusammenhänge,
- interpretieren Alltagssprache.

Computer:

- benötigen Daten und Regeln,
  - führen die vorgesehenen Schritte aus.
-

# Algorithmus und Programmiersprache

Problem



Lösungsidee



Algorithmus



|         |        |
|---------|--------|
| Scratch | Python |
|---------|--------|



# Scratch

Wenn grüne Flagge angeklickt  
sage [Willkommen!]

---

# Python

```
print("Willkommen!")
```

---

## Was bleibt gleich?

- Problem
- Lösungsidee
- Algorithmus

## Was ändert sich?

- Darstellung
  - Syntax
-

# Entwickeln heißt testen

Entwurf



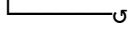
Test



Fehler / Schwäche



Verbesserung



# Abschluss

## Kannst du ...

- einen Algorithmus erkennen?
- Fehler erklären?
- einen Algorithmus entwickeln?
- eine Lösung testen?
- Algorithmus und Programmiersprache unterscheiden?

# Komplexaufgabe zur Algorithmierung

## Unser Ziel

Ein Problem selbstständig lösen.

---

# Nicht sofort programmieren

Problem



Analyse



Lösungsidee



Plan



Umsetzung



# Projektstart

## Drei Fragen

- Welches Problem lösen wir?
  - Was muss unsere Lösung können?
  - Was gehört bewusst nicht dazu?
-

# Muss oder Kann?

## Muss

Ohne diese Funktion ist das Projektziel nicht erreicht.

## Kann

Sinnvolle Erweiterung, wenn genügend Zeit vorhanden ist.

---

## **Gute Anforderungen**

Nicht:

Das Spiel soll cool sein.

Sondern:

Nach dem Start kann die Spielfigur mit vier Tasten bewegt werden.

---

# EVAS nutzen

Eingabe



Verarbeitung



Ausgabe



Speicherung

Welche Teile braucht euer Projekt?

---

# Mehr als eine Idee

Entwickelt mindestens zwei Lösungswege.

Vergleicht:

- Machbarkeit
  - Verständlichkeit
  - Testbarkeit
  - Zeitbedarf
-

# Version 1

## Klein anfangen

Was ist die kleinste Version, die bereits funktioniert?

---

# Inkrementell entwickeln

Version 0.1

↓

Version 0.2

↓

Version 0.3

Jede Version kann etwas mehr.

---

# Iterativ verbessern

Planen



Umsetzen



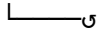
Testen



Bewerten



Verbessern



# Testfall

Ein Testfall enthält:

- Ausgangslage
  - Eingabe oder Aktion
  - erwartetes Ergebnis
  - tatsächliches Ergebnis
-

# Fremdtest

## Regel

Das testende Team erhält zunächst keine zusätzliche Erklärung.

Kann eine fremde Person eure Lösung verstehen?

---

# Fehler priorisieren

1. Kernfunktion
  2. Muss-Anforderungen
  3. Bedienbarkeit
  4. Kann-Anforderungen
  5. Aussehen
-

# Dokumentieren

Nicht:

Heute haben wir programmiert.

Sondern:

Was wurde entschieden, warum und mit welchem Ergebnis?

---

# Präsentieren

Zeigt:

1. Problem
  2. Ziel
  3. Lösungsweg
  4. Ergebnis
  5. Test
  6. wichtigste Verbesserung
  7. Erkenntnis
-

## **Abschlussfrage**

### **Was ist wichtiger?**

Ein kompliziertes unfertiges Projekt?

oder

eine kleine, getestete und nachvollziehbare Lösung?